# Programować każdy może

dr hab. Andrzej Zbrzezny, prof. UJD

Katedra Matematyki i Informatyki
Wydział Nauk Ścisłych, Przyrodniczych i Technicznych
Uniwersytet Jana Długosza w Częstochowie

Częstochowa, 17 marca 2026

Dzień Otwarty Uniwersytetu

# Programowanie

## Dlaczego warto uczyć się programowania komputerowego

- Programowanie promuje kreatywność, uczy precyzyjnego rozumowania i rozwiązywania problemów.
- Programista ma okazję stworzyć coś z niczego.
- Programowanie jest zabawne, chociaż bywa czasami trudne, a niekiedy frustrujące.
- Umiejętności zdobyte w czasie nauki programowania mogą być przydatne zarówno w szkole, jak i w pracy.
- Ponadto programowanie to świetny sposób na spędzanie wolnego czasu.
- A jednym z najlepszych języków do nauki programowania jest język Python.

# Programowanie

## Dlaczego warto używać Pythona

- Python od początku był tworzony z myślą, aby był łatwy do nauczenia. Programy w Pythonie jest łatwiej pisać, czytać, i rozumieć niż programy napisane w jakimkolwiek innym poważnym języku programowania.
- Python jest bezpłatny – można za darmo pobrać interpreter Pythona, jak również wiele zabawnych i użytecznych programów napisanych w Pythonie.
- Python jest świetnym językiem do nauki programowania, a jednocześnie jest używany przez tysiące profesjonalistów na całym świecie, w tym programistów w instytucjach takich jak Google, NASA, NSA, CIA i wielu innych.
- Python działa na różnych rodzajach komputerów. Jest dostępny dla komputerów z systemem Windows, Mac OS i komputerów z systemem Linux.

# Instalacja interpretera języka Python

## W systemie Windows

- Interpreter Pythona 3 można bezpłatnie pobrać ze strony `https://www.python.org/downloads/release/python-3143/`
- Pobieramy plik `python-3.14.3-amd64.exe` poprzez link `Windows installer (x86-64)`
- Instalujemy Pythona klikając na pobrany plik.
- Po zakończeniu instalacji możemy uruchomić interpreter Pythona wybierając z menu `Wszystkie programy` odpowiedni program: `Python 3.14.3 -> IDLE (Python GUI)`.

# Używanie interpretera Pythona

## W systemie Ubuntu Linux i innych dystrybucjach Linuksa

- W większości popularnych dystrybucji Linuksa Python jest domyślnie zainstalowany.
- Aby uruchomić interpreter Pythona 3 otwieramy terminal i wpisujemy polecenie: `python3`
- Pojawi się krótka informacja o interpreterze oraz znak zachęty:

```
>>>
```

- Od tej chwili możemy wpisywać instrukcje oraz wyrażenia języka Python, np.

```
>>> 1 + 2
3
>>>
```

# Używanie interpretera Pythona

## Importowanie modułów

- Aby móc korzystać z funkcji matematycznych należy zaimportować odpowiedni moduł:

```
>>> import math
```

- Od tej chwili możemy używać funkcji zdefiniowanych w tym module, np.

```
>>> math.sqrt(2)
1.4142135623730951
```

- Aby uzyskać pomoc na temat funkcji i stałych zdefiniowanych w module `math` należy użyć funkcji `help`:

```
>>> help(math)
```

# Używanie interpretera Pythona

## Przykład (Wykorzystanie funkcji `print`)

```
>>> print("Witajcie uczniowie!")
Witajcie uczniowie!
>>>
>>> print(2 + 3)
5
>>>
>>> print("2 + 3 = ", 2 + 3)
2 + 3 = 5
>>>
>>> print(2, "+", 3, "=", 2 + 3)
2 + 3 = 5
>>>
```

## Przykłady (Liczby i operatory)

```
>>> 3 * 8
24
>>> 20 // 3
6
>>> 20 % 3
2
>>> 20 / 3
6.666666666666667
>>> 2.71 + 1
3.71
>>> 2 ** 6
64
```

## Przykłady (Operatory relacyjne)

```
>>> 5 < 8
True
>>> 5 <= 8
True
>>> 5 == 8
False
>>> 5 != 8
True
>>> 5 > 8
False
>>> 5 >= 8
False
```

## Przykłady (Wbudowana funkcja type)

```
>>> type(2026)
<class int>
>>> type("2026")
<class str>
>>> type(3.14)
<class float>
>>> type(1 > 0)
<class bool>
>>> type(1 + 2j)
<class complex>
>>> type(None)
<class NoneType>
```

# Używanie interpretera Pythona

## Przykład (Zakres liczb całkowitych ograniczony tylko pamięcią)

```
>>> 2**2026
770497466833853895681259076325177001010038916342627 6
214630686146622813973156903003024387696152428022310 4
281332306618736968325517536380102861137753454323489 9
535915311987423803609614318618527701063062134704209 9
849766920141850142420807323001465434115045209323370 9
421885115832271317341484567183381078538728679470811 6
850606640396611332292236437070247082138903768588497 1
095480759388492753138617074551246849102286511884594 8
501734393495555761223558535347947353394587724760120 2
146642466073268059661429570468031637157024876038978 9
353846852386712703279382030103072484861279043062503 2
436002060123932247515186204561259888 64
```

# Podstawy języka Python

## Skrypty w języku Python

- Skrypt Pythona to plik tekstowy, zazwyczaj z rozszerzeniem `.py`, zawierający program zapisany w języku Python.

## Przykład (Przykładowy skrypt `hello.py`)

```
print("Witajcie uczniowie!")
```

## Przykład (Przykładowy skrypt z użyciem funkcji `main`)

```
def main():
    print("Witajcie uczniowie!")

main()
```

## Przykład (Skrypt obliczający wyniki podstawowych działań)

```
#!/usr/bin/python3

""" arytmetyczne.py """

def main():
    a = float(input("Podaj pierwszą liczbę: "))
    b = float(input("Podaj pierwszą liczbę: "))
    print("Suma wczytanych liczb wynosi", a + b)
    print("Róznica wczytanych liczb wynosi", a - b)
    print("Iloczyn wczytanych liczb wynosi", a * b)
    print("Iloraz wczytanych liczb wynosi", a / b)

main()
```

## Zmienne

- W Pythonie zmienna jest referencją do obiektu. Zamiast słowa referencja używa się też słowa odniesienie.
- Aby utworzyć zmienną oraz nadać jej wartość należy użyć instrukcji przypisania:

```
zmienna = wyrażenie
```

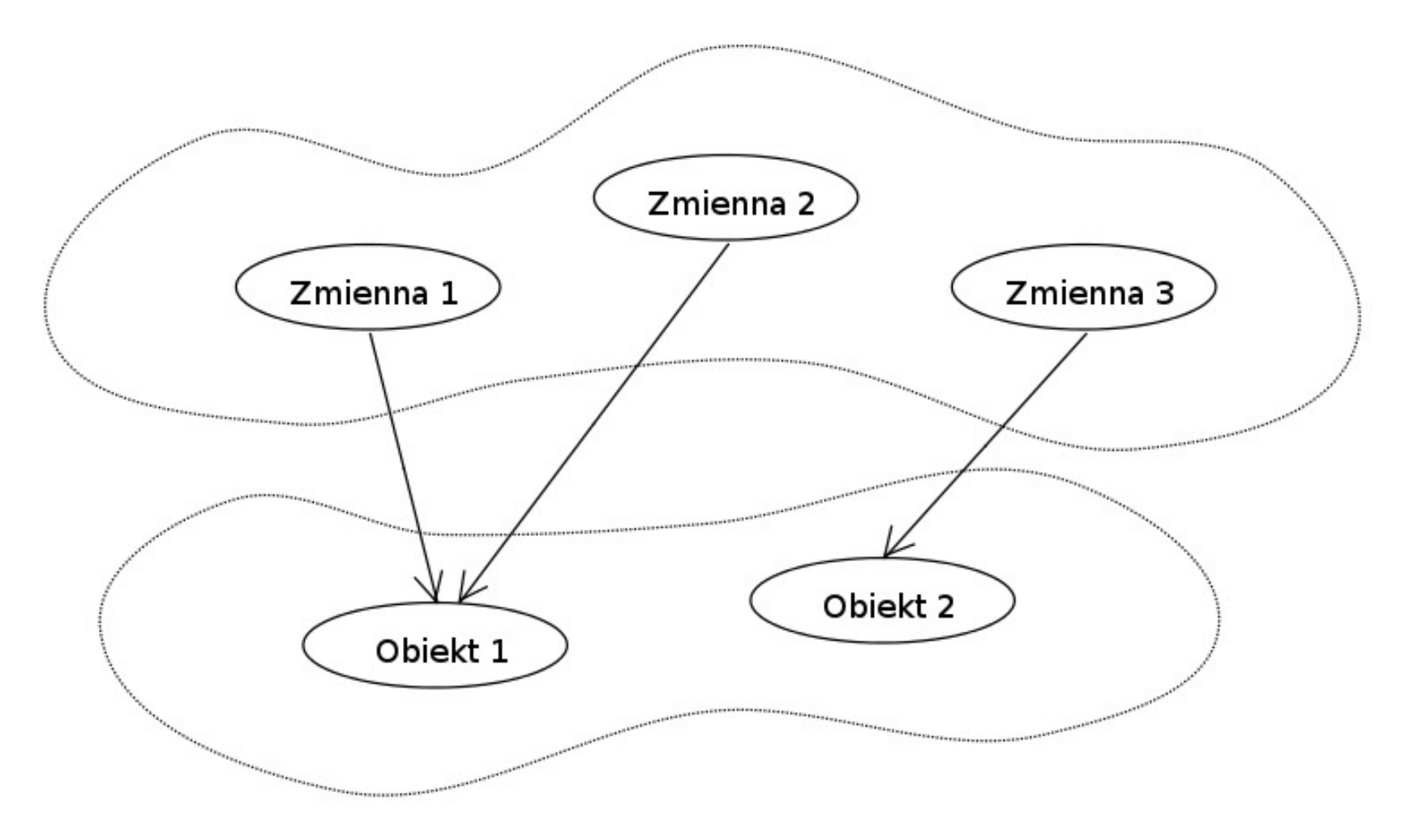
- Powyższa instrukcja przypisuje do zmiennej `zmienna` odniesienie do obiektu utworzonego w wyniku obliczenia wartości wyrażenia `wyrażenie`.
- Wartość może być przypisana jednocześnie wielu zmiennym, np.

```
a = b = c = 0
```

- Możliwe są też jednoczesne przypisania:

```
a, b, c = 1, 2, 3
```

## Referencje i obiekty



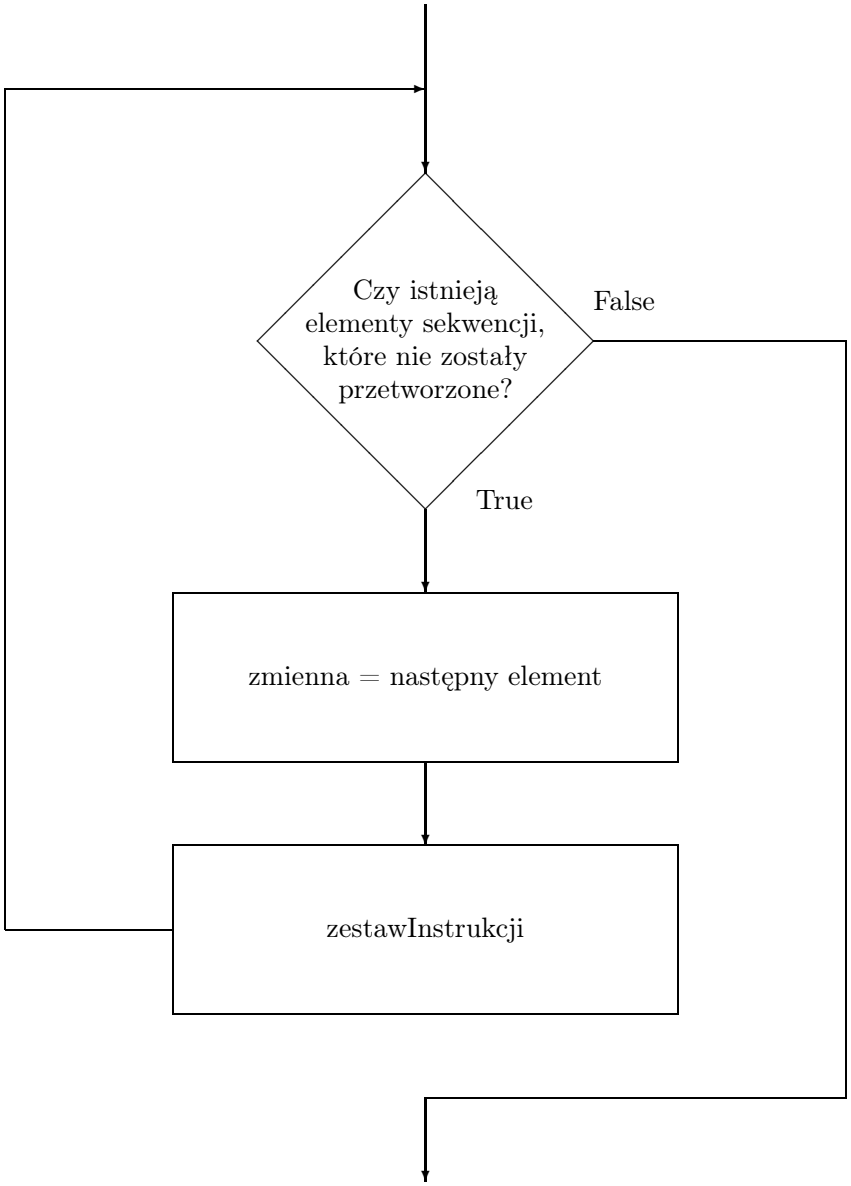
## Prosta postać instrukcji iteracyjnej `for` (`dla`)

- Instrukcja iteracyjna `for` jest używana do iterowania po elementach sekwencji.
- Prosta postać instrukcji iteracyjnej `for` jest następująca:

```
for zmienna in wyrażenie:
    zestawInstrukcji
```

- `zestawInstrukcji` to ciąg instrukcji, które umieszczone są w kolejnych liniach, przy czym każda linia jest wcięta w stosunku do linii zaczynającej się od słowa kluczowego **`for`** o tę samą liczbę spacji.

# Schemat blokowy instrukcji iteracyjnej `for`



## Przykłady (Funkcja wbudowana range)

```
>>> list(range(10))
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> list(range(1, 11))
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
>>> list(range(0, 30, 5))
[0, 5, 10, 15, 20, 25]
>>> list(range(0, 10, 3))
[0, 3, 6, 9]
>>> list(range(0, -10, -1))
[0, -1, -2, -3, -4, -5, -6, -7, -8, -9]
>>> list(range(0))
[]
```

## Przykład (Program z użyciem instrukcji iteracyjnej for)

```
"""Oblicza sumę liczb naturalnych od 1 do n"""

def main():
    n = int(input("Podaj liczbę naturalną: "))
    s = 0
    for j in range(1, n + 1):
        s = s + j
    print("Suma liczb od 1 do", n, "wynosi", s)

main()
```

## Przykład (n-ta liczba w ciągu Fibonacciego)

```
def fib(n):
    a, b = 0, 1
    for j in range(n):
        a, b = b, a + b
    return a

def main():
    n = int(input("Podaj liczbę naturalną: "))
    print("fib(" + str(n) + ") =", fib(n))

main()
```

## Grafika żółwia

- Grafika żółwia jest popularnym sposobem na wprowadzenie programowania dla dzieci.
- Była ona częścią oryginalnego języka programowania Logo zaprojektowanego w 1966 roku przez Seymoura Paperta pracującego w Massachusetts Institute of Technology.
- Aby w systemie Ubuntu móc używać grafiki żółwia w Pythonie 3 należy zainstalować pakiet `python3-tk`:
  ```
  $ sudo apt-get install python3-tk
  ```
- Grafika żółwia w Pythonie dostępna jest poprzez moduł `turtle`:
  ```
  import turtle
  ```

## Example (Grafika żółwia – program square4slides.py)

```
import turtle

def draw_multicolor_square(t, length):
    for c in ["red", "purple", "hotpink", "blue"]:
        t.color(c); t.forward(length); t.left(90)

def main():
    wn = turtle.Screen(); wn.bgcolor("lightgreen")
    tess = turtle.Turtle(); tess.pensize(3)
    for length in range(20, 200, 10):
        draw_multicolor_square(tess, length)
        tess.forward(10); tess.right(18)
    wn.mainloop()

main()
```

# Podstawy języka Python– grafika żółwia

## Przykłady

- `squares.py` – program rysujący kwadraty
- `circle.py` – program rysujący okrąg
- `wheels.py` – program rysujący kolorowe koła
- `sun.py` – program rysujący słońce

# Finis coronat opus

Dziękuję za uwagę ;-)